HW1 Report: Distributed Computing Basics

Amir Mahdi Zaryoun – 810100157

Amir Hosein Sabahi – 810101556

Environment

The implementation and execution were done on a fresh Ubuntu 24.04 VM brought up on Sangfor virtualization platform.

Environment Summary:

Item	Value
Operating System	Ubuntu 24.04.4 LTS
Kernel	Linux 6.8.0-111-generic
CPU cores	2
Memory	4 GB
Disk	50 GB mounted on /
Docker version	Docker 29.4.3
Go version	go1.22.2 linux/amd64
Go proxy	https://package-mirror.darvagcloud.com/go
APT repository	https://mirror.darvagcloud.com/ubuntu
Docker CE repository	https://mirror.darvagcloud.com/docker-ce-deb
Docker container registry	https://docker.darvagcloud.com

The implementation uses only Go standard-library packages. No external Go dependency was added.

Table of Contents

1. Overview

This homework contains three connected parts.

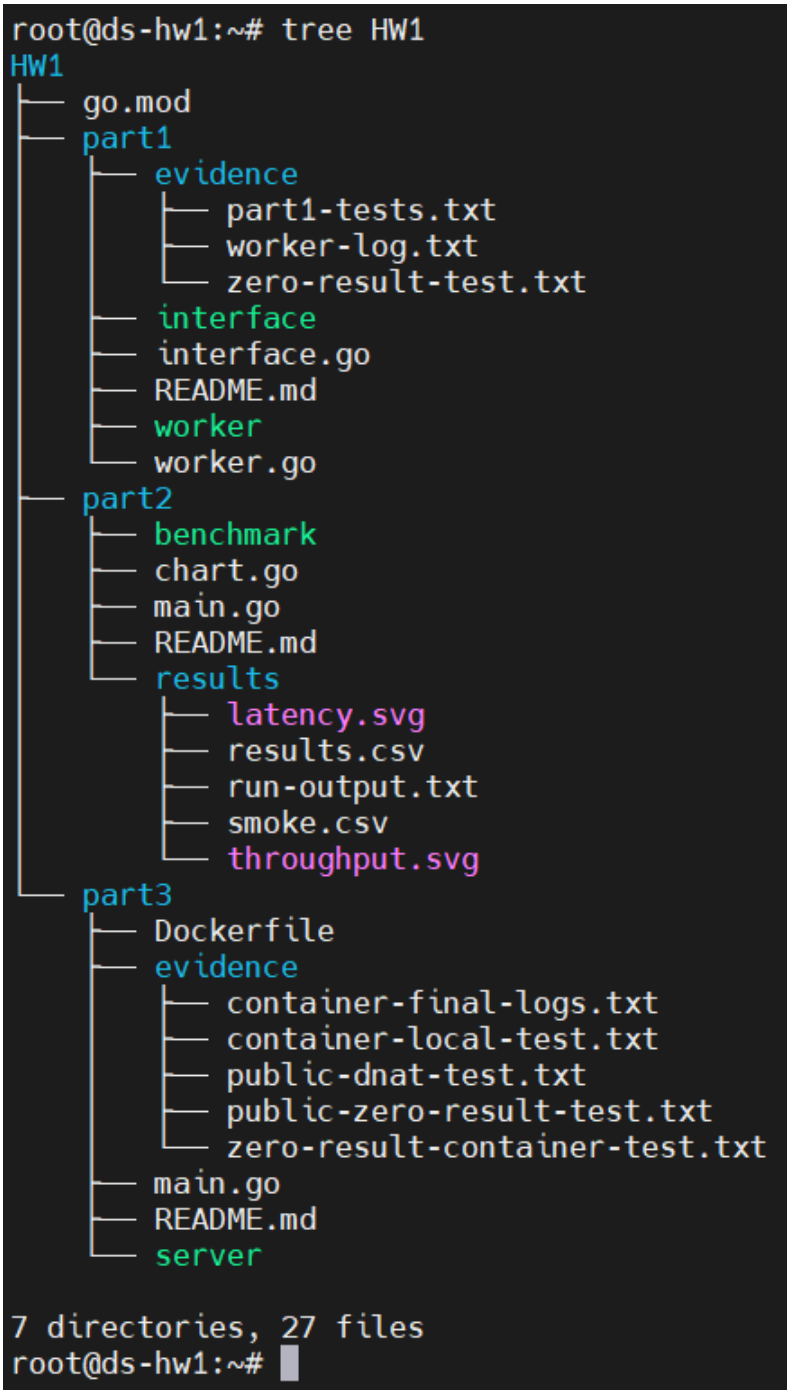
In Part 1, a simple computation service was implemented using two independent processes communicating through Linux named pipes. The Interface process receives user input and sends requests to the Worker process. The Worker process remains alive, listens for requests, computes results, and sends responses back.

In Part 2, the behavior of Go goroutines was studied by changing the number of goroutines, workload type, and GOMAXPROCS. The goal was to observe the effect of scheduling, CPU contention, synchronization, and increased concurrency on throughput and latency.

In Part 3, the same computation idea was implemented as an HTTP service, containerized using Docker, executed inside the VM, and exposed outside the VM through Docker port publishing and Sangfor firewall DNAT.

2. Project Structure

The final project structure is (report.pdf excluded):



The compiled binaries are included in the final project folder:

- part1/worker
- part1/interface
- part2/benchmark
- part3/server

3. Part 1: IPC Between Two Independent Processes Using Named Pipes

3.1. Architecture

Part 1 consists of two independent Go programs:

- worker.go
- interface.go

The Worker process is a persistent computation service. It creates and listens on Linux named pipes. The Interface process acts as the user-facing client. It sends one request to the Worker and waits for one response.

Two named pipes are used:

- /tmp/hw1_request.pipe
- /tmp/hw1_response.pipe

The request pipe carries messages from Interface to Worker. The response pipe carries messages from Worker back to Interface.

This design keeps the two roles separated into different operating-system processes. The Worker is not restarted for each request; it remains alive and processes multiple sequential requests.

3.2. IPC Protocol

The request format is: OP A B

Where OP is the operation name, A is the first numeric operand and B is the second numeric operand.

Example requests:

- ADD 5 7
- SUB 10 3
- MUL 4 6
- DIV 8 2

The response format is JSON. JSON was selected because it makes the protocol explicit, structured, and easier to debug.

Successful response example: {"ok":true,"operation":"ADD","a":5,"b":7,"result":12}

Error response example: {"ok":false,"operation":"DIV","a":8,"b":0,"error":"division_by_zero"}

3.3. Supported Operations

The required operations are: ADD, SUB, MUL, DIV

Additional operations were implemented for distinction: MOD, POW, MAX, MIN

These extra operations show that the protocol and computation logic can be extended without changing the overall IPC architecture.

3.4. Error Handling

The implementation handles the following error cases:

Error Case	**Example**
Unknown operation	ABC 1 2
Invalid argument count	ADD 1
Non-numeric first operand	ADD x 2
Non-numeric second operand	ADD 1 y
Division by zero	DIV 8 0
Modulo by zero	MOD 8 0
Missing named pipe	Interface reports missing Worker/pipe
Worker not running	Interface reports pipe/open error
Unexpected pipe read/write error	Error is returned instead of silent crash

3.5. Build and Run Commands

Build commands:

go build -o part1/worker part1/worker.go
go build -o part1/interface part1/interface.go

Correct execution order: ./part1/worker

Then, in another terminal: ./part1/interface ADD 5 7

The Interface also supports interactive mode: ./part1/interface

3.6. Sample Outputs

```
root@ds-hw1:~/HW1# ./part1/interface ADD 5 7
{"ok":true,"operation":"ADD","a":5,"b":7,"result":12}
root@ds-hw1:~/HW1# ./part1/interface SUB 10 3
{"ok":true,"operation":"SUB","a":10,"b":3,"result":7}
root@ds-hw1:~/HW1# ./part1/interface MUL 4 6
{"ok":true,"operation":"MUL","a":4,"b":6,"result":24}
root@ds-hw1:~/HW1# ./part1/interface DIV 8 2
{"ok":true,"operation":"DIV","a":8,"b":2,"result":4}
root@ds-hw1:~/HW1# ./part1/interface MOD 10 3
{"ok":true,"operation":"MOD","a":10,"b":3,"result":1}
root@ds-hw1:~/HW1# ./part1/interface POW 2 8
{"ok":true,"operation":"POW","a":2,"b":8,"result":256}
root@ds-hw1:~/HW1# ./part1/interface MAX 12 9
{"ok":true,"operation":"MAX","a":12,"b":9,"result":12}
root@ds-hw1:~/HW1# ./part1/interface MIN 12 9
{"ok":true,"operation":"MIN","a":12,"b":9,"result":9}
root@ds-hw1:~/HW1# ./part1/interface DIV 8 0
{"ok":false,"operation":"DIV","a":8,"b":0,"error":"division_by_zero"}
root@ds-hw1:~/HW1# ./part1/interface ABC 1 2
{"ok":false,"operation":"ABC","a":1,"b":2,"error":"unknown_operation"}
root@ds-hw1:~/HW1# ./part1/interface ADD 1
{"ok":false,"operation":"","a":0,"b":0,"error":"invalid_argument_count"}
root@ds-hw1:~/HW1# ./part1/interface ADD x 2
{"ok":false,"operation":"","a":0,"b":0,"error":"invalid_number_a"}
root@ds-hw1:~/HW1# ./part1/interface ADD 1 y
{"ok":false,"operation":"","a":0,"b":0,"error":"invalid_number_b"}
root@ds-hw1:~/HW1# ./part1/interface SUB 5 5
{"ok":true,"operation":"SUB","a":5,"b":5,"result":0}
root@ds-hw1:~/HW1#
```

3.7. Worker Persistence Evidence

The Worker log showed that the Worker started once and then handled multiple requests:

```
root@ds-hw1:~/HW1# cat part1/evidence/worker-log.txt
2026/05/08 18:24:19.694172 worker started
2026/05/08 18:24:19.694385 request pipe: /tmp/hw1_request.pipe
2026/05/08 18:24:19.694394 response pipe: /tmp/hw1_response.pipe
2026/05/08 18:24:19.694396 waiting for requests...
2026/05/08 18:24:26.159920 received request: "DIV 8 0"
2026/05/08 18:24:26.164747 received request: "ADD 0 5"
2026/05/08 18:24:26.167847 received request: "SUB 5 0"
2026/05/08 18:24:26.170891 received request: "ABC 1 2"
2026/05/08 18:25:33.023543 received request: "ADD 5 7"
2026/05/08 18:25:33.027856 received request: "SUB 10 3"
2026/05/08 18:25:33.032324 received request: "MUL 4 6"
2026/05/08 18:25:33.036284 received request: "DIV 8 2"
2026/05/08 18:25:33.039456 received request: "MOD 10 3"
2026/05/08 18:25:33.042215 received request: "POW 2 8"
2026/05/08 18:25:33.045007 received request: "MAX 12 9"
2026/05/08 18:25:33.047529 received request: "MIN 12 9"
2026/05/08 18:25:33.049932 received request: "DIV 8 0"
2026/05/08 18:25:33.052331 received request: "ABC 1 2"
2026/05/08 18:25:33.054727 received request: "ADD 1"
2026/05/08 18:25:33.057195 received request: "ADD x 2"
2026/05/08 18:25:33.059516 received request: "ADD 1 y"
root@ds-hw1:~/HW1#
```

This confirms that the Worker remained alive and processed multiple sequential requests without restarting.

4. Part 2: Goroutine Scheduling and Performance Experiment

4.1. Goal

The goal of Part 2 was to observe how increasing concurrency affects performance in Go programs.

In Go, goroutines are managed by the Go runtime scheduler. The experiment does not directly implement operating-system-level context switching. Instead, it observes the effects of:

- Go scheduler behavior
- CPU contention
- Increasing number of goroutines

- Synchronization overhead
- Different values of GOMAXPROCS

4.2. Experiment Parameters

Three parameters were varied:

1. Number of goroutines
2. Workload type
3. GOMAXPROCS

The goroutine counts were: 1, 2, 4, 8, 16, 32, 64

The tested GOMAXPROCS values were: 1, 2, runtime.NumCPU(), 4

On this VM, runtime.NumCPU() was equal to 2. Since this duplicated the required value 2, the additional value GOMAXPROCS=4 was tested as an oversubscription case.

Each goroutine executed 500 jobs in the final benchmark.

4.3. Workload Types

Two workload types were implemented.

CPU-bound workload

Each goroutine repeatedly performs numeric computation using a loop with floating-point operations. This workload mainly stresses CPU execution. It is useful for observing how much benefit is gained when GOMAXPROCS is increased from 1 to the number of available CPU cores.

Mixed workload

Each goroutine performs a smaller amount of computation and then enters a shared critical section protected by a mutex. This workload adds synchronization overhead and makes contention more visible.

4.4. Metrics

The program records the following metrics for each configuration:

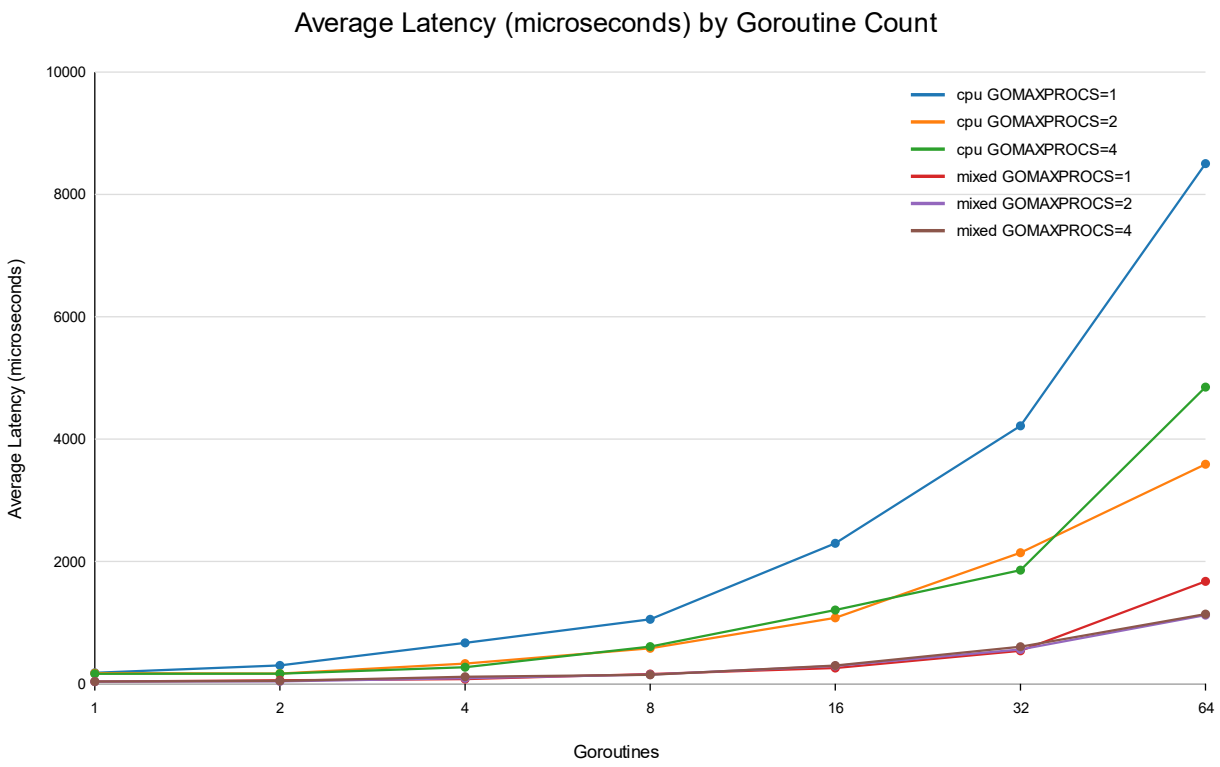
Metric	Meaning
Workload	CPU-bound or mixed
GOMAXPROCS	Maximum number of OS threads executing Go code
Goroutines	Number of concurrent goroutines
Jobs per worker	Jobs executed by each goroutine

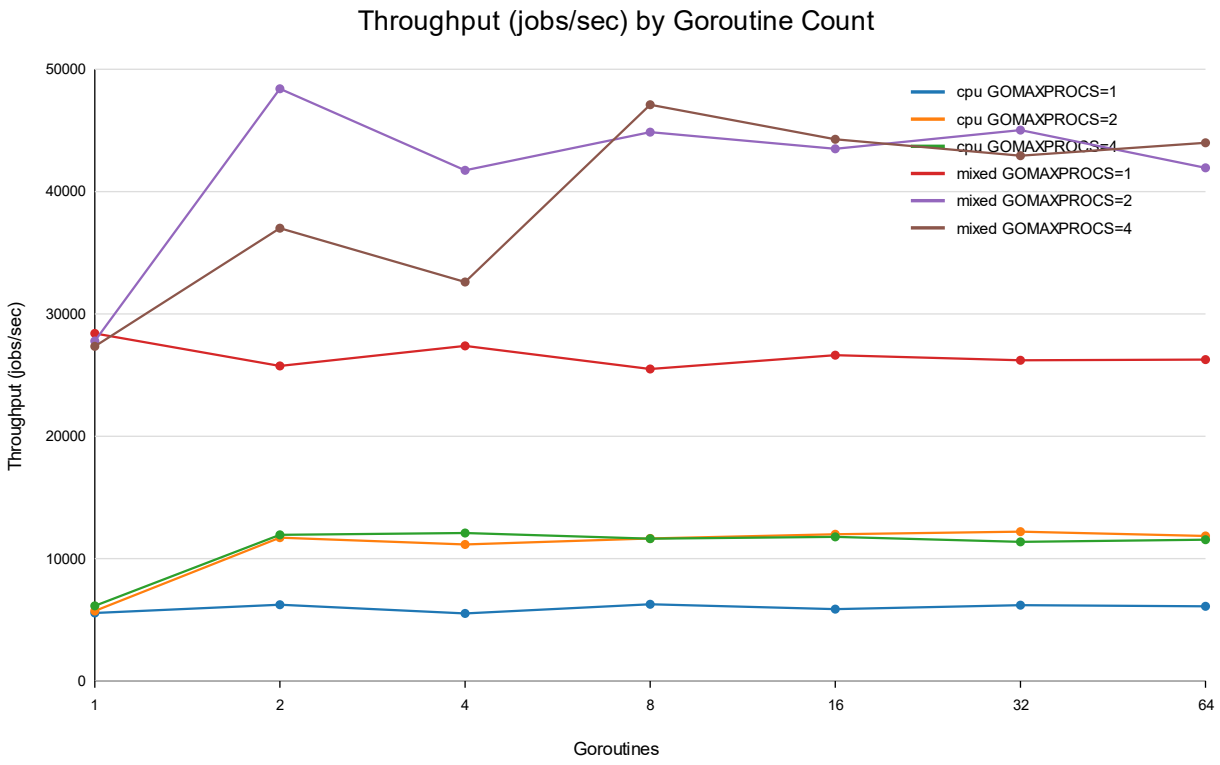
Total jobs	Goroutines * jobs_per_worker
Total time	Total benchmark duration in milliseconds
Throughput	Jobs completed per second
Average latency	Average time per job in microseconds
Minimum latency	Fastest job latency
Maximum latency	Slowest job latency
Standard deviation	Variation in job latency

The complete result table was saved in part2/results/run-output.txt

Charts were generated as SVG files: part2/results/throughput.svg, part2/results/latency.svg

Charts below:


Average Latency (microseconds) by Goroutine Count
Average Latency (microseconds)
10000
8000
6000
4000
2000
0
1
2
4
8
16
32
64
Goroutines
cpu GOMAXPROCS=1
cpu GOMAXPROCS=2
cpu GOMAXPROCS=4
mixed GOMAXPROCS=1
mixed GOMAXPROCS=2
mixed GOMAXPROCS=4


Throughput (jobs/sec) by Goroutine Count
Throughput (jobs/sec)
50000
40000
30000
20000
10000
0
1
2
4
8
16
32
64
Goroutines
cpu GOMAXPROCS=1
cpu GOMAXPROCS=2
cpu GOMAXPROCS=4
mixed GOMAXPROCS=1
mixed GOMAXPROCS=2
mixed GOMAXPROCS=4

4.5. Build and Run Commands

Build: go build -o part2/benchmark part2/main.go

Smoke test: ./part2/benchmark -jobs 50 -out part2/results/smoke.csv

Full benchmark: ./part2/benchmark -jobs 500 -out part2/results/results.csv | tee part2/results/run-output.txt

Generate charts: go run part2/chart.go part2/results/results.csv

4.6. Selected Results Table

The following table contains representative rows from the full benchmark. The complete table is available in part2/results/results.csv.

Workload	GOMAXPROCS	Goroutines	Total Time (ms)	Throughput (jobs/sec)	Avg Latency (µs)
cpu	1	1	89.925	5560.180	179.172
cpu	1	2	160.378	6235.238	300.496
cpu	1	4	362.717	5513.938	668.107
cpu	1	8	638.473	6264.938	1053.800
cpu	1	16	1362.649	5870.915	2296.264
cpu	1	32	2583.887	6192.220	4216.907
cpu	1	64	5245.285	6100.717	8504.560
cpu	2	1	87.429	5718.922	174.110
cpu	2	2	85.398	11709.846	166.931
cpu	2	4	179.330	11152.840	329.640
cpu	2	8	343.478	11645.575	579.941
cpu	2	16	667.365	11987.440	1077.210
cpu	2	32	1311.000	12204.419	2141.974
cpu	2	64	2699.525	11853.936	3589.205
cpu	4	1	81.449	6138.782	162.324
cpu	4	2	83.689	11948.914	165.442
cpu	4	4	165.390	12092.310	269.810
cpu	4	8	343.834	11633.501	606.779
cpu	4	16	679.290	11776.990	1206.150
cpu	4	32	1407.159	11370.425	1857.213
cpu	4	64	2771.139	11547.595	4849.767
mixed	1	1	17.602	28404.963	34.778
mixed	1	2	38.835	25749.661	57.330
mixed	1	4	73.010	27393.150	75.460
mixed	1	8	156.865	25499.614	158.454
mixed	1	16	300.370	26633.400	257.980

mixed	1	32	610.486	26208.617	538.379
mixed	1	64	1218.006	26272.437	1673.349
mixed	2	1	17.991	27791.272	35.514
mixed	2	2	20.659	48403.043	39.774
mixed	2	4	47.910	41741.240	89.820
mixed	2	8	89.164	44861.112	149.544
mixed	2	16	183.900	43501.860	290.800
mixed	2	32	355.350	45025.933	561.783
mixed	2	64	762.939	41943.011	1122.238
mixed	4	1	18.282	27347.826	36.148
mixed	4	2	27.022	37006.727	47.112
mixed	4	4	61.322	32614.286	114.332
mixed	4	8	84.929	47097.706	145.708
mixed	4	16	180.681	44276.820	298.335
mixed	4	32	372.644	42936.387	604.790
mixed	4	64	727.369	43994.170	1139.596

4.7. Analysis

With GOMAXPROCS=1, the CPU-bound workload had throughput around 5500 to 6300 jobs/sec. Increasing the number of goroutines did not significantly increase throughput because only one operating-system thread was allowed to execute Go code at a time. More goroutines mostly increased scheduling pressure and average latency.

With GOMAXPROCS=2, CPU-bound throughput almost doubled in several configurations. For example, with 2 goroutines, throughput increased from about 6235 jobs/sec at GOMAXPROCS=1 to about 11710 jobs/sec at GOMAXPROCS=2. This matches the VM having 2 vCPUs.

With GOMAXPROCS=4, throughput did not improve proportionally compared with GOMAXPROCS=2. This is expected because the VM only had 2 vCPUs. Allowing more Go scheduler threads than physical CPU resources can increase scheduling overhead without adding real CPU capacity.

For the mixed workload, throughput was higher than the CPU-bound workload because each job performed less computation. However, average latency increased as the number of goroutines increased. This is caused by mutex contention and scheduling overhead. At high goroutine counts, goroutines spend more time waiting for CPU time or waiting to enter the critical section.

Increasing goroutine count was not always beneficial. In the CPU-bound workload, after enough goroutines were available to use the available CPU cores, adding more goroutines

mainly increased latency. In the mixed workload, the best throughput occurred at moderate concurrency; beyond that point, synchronization and scheduling overhead became more visible.

The strongest scheduling and switching effects were visible in high-goroutine configurations, especially where average latency and maximum latency increased. The mixed workload also showed synchronization overhead because goroutines competed for a shared mutex.

4.8. Answers to Required Analysis Questions

1. What happened to execution time as the number of goroutines increased?

Total execution time increased as the number of goroutines increased because the benchmark used a fixed number of jobs per goroutine. Therefore, more goroutines also meant more total work. However, the important comparison is throughput and latency. Throughput improved when more CPU capacity was made available through GOMAXPROCS=2, but latency increased at high goroutine counts.

2. Did increasing the number of goroutines always increase throughput?

No. Increasing goroutine count did not always increase throughput. In the CPU-bound workload with GOMAXPROCS=1, throughput stayed almost flat because only one CPU thread could execute Go code. In the mixed workload, excessive goroutine counts increased latency due to synchronization and scheduling overhead.

3. What was the difference between GOMAXPROCS=1 and GOMAXPROCS=runtime.NumCPU()?

On this VM, runtime.NumCPU() was 2. Moving from GOMAXPROCS=1 to GOMAXPROCS=2 allowed Go code to execute on two OS threads, matching the VM's two vCPUs. This significantly improved CPU-bound throughput. For example, with 2 CPU-bound goroutines, throughput increased from about 6235 jobs/sec to about 11710 jobs/sec.

4. In which workload were scheduling and switching effects more visible?

Scheduling effects were visible in both workloads, but the mixed workload made synchronization overhead more obvious because goroutines also competed for a mutex. The CPU-bound workload showed CPU contention clearly, while the mixed workload showed both scheduling and lock contention effects.

5. From what point was increasing concurrency no longer useful?

For the CPU-bound workload, increasing goroutines beyond the number needed to keep the available vCPUs busy did not provide major throughput improvement. With GOMAXPROCS=2, throughput was already close to its useful range at low to moderate goroutine counts. Higher goroutine counts mostly increased average latency. For the mixed workload, high goroutine counts increased waiting time around the shared mutex and raised latency.

5. Part 3: HTTP Service, Docker, and VM Deployment

5.1. Architecture

Part 3 implements the computation service as an HTTP server in Go. The server exposes two endpoints:

GET /health
GET /compute

The service was compiled into a Docker image and executed inside the Ubuntu VM. Docker published container port 8080 to VM port 8080. A Sangfor firewall DNAT rule was then configured so the service was reachable from outside the VM.

5.2. HTTP Endpoints

Health Endpoint

Request: GET /health
Example response: {"status":"ok","timestamp":"2026-05-08T19:51:16Z"}

Compute Endpoint

Request format: GET /compute?op=<operation>&a=<number>&b=<number>

Example: GET /compute?op=add&a=5&b=7
Response: {"operation":"add","a":5,"b":7,"result":12}

5.3. Supported Operations

Required operations:

- add
- sub
- mul
- div

Extra operations:

- mod
- pow
- max
- min

5.4. HTTP Error Handling

The service handles:

Error case	Example
Missing parameters	/compute?op=add&a=5
Invalid operation	/compute?op=bad&a=1&b=2
Non-numeric a	/compute?op=add&a=x&b=2
Non-numeric b	/compute?op=add&a=1&b=y
Division by zero	/compute?op=div&a=8&b=0
Modulo by zero	/compute?op=mod&a=8&b=0
Wrong method	POST /compute?op=add&a=1&b=2

Wrong method request: POST /compute?op=add&a=1&b=2
Response: {"error":"method_not_allowed"}

5.5. Build and Run Without Docker

Build command: go build -o part3/server part3/main.go

Run command: PORT=8080 ./part3/server

5.6. Dockerfile Design

The Dockerfile uses a multi-stage build.

The first stage uses a Go image to compile the service binary. The second stage uses a Debian slim image to run the compiled binary.

The Docker images are pulled from the Darvag Docker mirror:

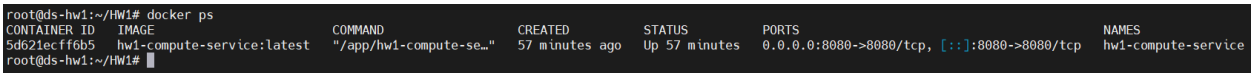
- docker.darvagcloud.com/library/golang:1.22-bookworm
- docker.darvagcloud.com/library/debian:bookworm-slim

Build command: docker build -t hw1-compute-service:latest -f part3/Dockerfile .

Run command: docker run -d --name hw1-compute-service -p 8080:8080 hw1-compute-service:latest

5.7. Docker Execution Evidence

Docker build completed successfully. The container was started with port publishing:



This shows that the service inside the container was published to VM port 8080.

5.8. Local Container Test Inside VM

Health and compute request from inside the VM:

```
root@ds-hw1:~/HW1# curl -i http://127.0.0.1:8080/health
HTTP/1.1 200 OK
Content-Type: application/json
Date: Fri, 08 May 2026 20:01:16 GMT
Content-Length: 51

{"status":"ok","timestamp":"2026-05-08T20:01:16Z"}
root@ds-hw1:~/HW1# curl -i "http://127.0.0.1:8080/compute?op=add&a=5&b=7"
HTTP/1.1 200 OK
Content-Type: application/json
Date: Fri, 08 May 2026 20:01:22 GMT
Content-Length: 44

{"operation":"add","a":5,"b":7,"result":12}
root@ds-hw1:~/HW1# curl -i "http://127.0.0.1:8080/compute?op=div&a=8&b=0"
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Fri, 08 May 2026 20:01:29 GMT
Content-Length: 59

{"operation":"div","a":8,"b":0,"error":"division_by_zero"}
root@ds-hw1:~/HW1# curl -i "http://127.0.0.1:8080/compute?op=pow&a=2&b=8"
HTTP/1.1 200 OK
Content-Type: application/json
Date: Fri, 08 May 2026 20:01:36 GMT
Content-Length: 45

{"operation":"pow","a":2,"b":8,"result":256}
root@ds-hw1:~/HW1# curl -i "http://127.0.0.1:8080/compute?op=sub&a=5&b=5"
HTTP/1.1 200 OK
Content-Type: application/json
Date: Fri, 08 May 2026 20:01:44 GMT
Content-Length: 43

{"operation":"sub","a":5,"b":5,"result":0}
root@ds-hw1:~/HW1#
```

5.9. Host to VM Access Through DNAT

The VM private address was 192.168.1.13/24

The Docker container exposed the service on VM port 8080: 0.0.0.0:8080 -> 8080/tcp

A DNAT rule was configured on the Sangfor firewall:

Field	Value
Public IP	89.38.215.235
Public port	18080
Internal target	192.168.1.13
Internal port	8080
Protocol	TCP

This maps 89.38.215.235:18080 -> 192.168.1.13:8080

External host test from laptop:

```
amir@silverbot:~$ curl -i http://89.38.215.235:18080/health
HTTP/1.1 200 OK
Content-Type: application/json
Date: Fri, 08 May 2026 20:05:52 GMT
Content-Length: 51

{"status":"ok","timestamp":"2026-05-08T20:05:52Z"}
amir@silverbot:~$ curl -i "http://89.38.215.235:18080/compute?op=add&a=5&b=7"
HTTP/1.1 200 OK
Content-Type: application/json
Date: Fri, 08 May 2026 20:05:58 GMT
Content-Length: 44

{"operation":"add","a":5,"b":7,"result":12}
amir@silverbot:~$ curl -i "http://89.38.215.235:18080/compute?op=div&a=8&b=0"
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Fri, 08 May 2026 20:06:08 GMT
Content-Length: 59

{"operation":"div","a":8,"b":0,"error":"division_by_zero"}
amir@silverbot:~$ curl -i "http://89.38.215.235:18080/compute?op=sub&a=5&b=5"
HTTP/1.1 200 OK
Content-Type: application/json
Date: Fri, 08 May 2026 20:06:17 GMT
Content-Length: 43

{"operation":"sub","a":5,"b":5,"result":0}
amir@silverbot:~$
```

This confirms that the final container image was reachable through the public DNAT path.

5.10. Container Logs Showing External Requests

The container logs showed requests reaching the Go service:

```
root@ds-hw1:~/HW1# docker logs hw1-compute-service
2026/05/08 19:02:15 starting HW1 compute service on :8080
2026/05/08 19:02:15 GET /compute?op=sub&a=5&b=5 from=172.17.0.1:35512 duration=70.527µs
2026/05/08 19:03:41 GET /compute?op=sub&a=5&b=5 from=172.17.0.1:35842 duration=88.877µs
2026/05/08 19:03:45 GET /compute?op=sub&a=5&b=5 from=89.38.215.235:53312 duration=49.019µs
2026/05/08 19:04:39 GET /compute?op=sub&a=5&b=5 from=89.38.215.235:57362 duration=41.681µs
2026/05/08 19:51:16 GET /health from=89.38.215.235:54516 duration=199.694µs
2026/05/08 20:01:16 GET /health from=172.17.0.1:49644 duration=18.315µs
2026/05/08 20:01:22 GET /compute?op=add&a=5&b=7 from=172.17.0.1:33804 duration=57.258µs
2026/05/08 20:01:29 GET /compute?op=div&a=8&b=0 from=172.17.0.1:34966 duration=178.62µs
2026/05/08 20:01:36 GET /compute?op=pow&a=2&b=8 from=172.17.0.1:34980 duration=43.273µs
2026/05/08 20:01:44 GET /compute?op=sub&a=5&b=5 from=172.17.0.1:50992 duration=39.908µs
2026/05/08 20:05:52 GET /health from=89.38.215.235:64356 duration=20.636µs
2026/05/08 20:05:58 GET /compute?op=add&a=5&b=7 from=89.38.215.235:64360 duration=142.618µs
2026/05/08 20:06:08 GET /compute?op=div&a=8&b=0 from=89.38.215.235:64372 duration=67.346µs
2026/05/08 20:06:17 GET /compute?op=sub&a=5&b=5 from=89.38.215.235:64332 duration=65.056µs
root@ds-hw1:~/HW1#
```

The external source address in the logs confirms that public requests reached the containerized service.

6. Challenges and Solutions

6.1. Part 1 Challenges

One challenge in Part 1 was designing the pipe communication so that the Worker remained alive and could process multiple requests. This was solved by using two named pipes and keeping the Worker in a continuous read loop.

Another issue was making error responses clear. JSON responses were used so each result has an explicit success flag, operation, operands, result, or error code.

A later improvement fixed zero-result responses. Initially, successful results equal to zero could be omitted from JSON because of omitempty. This was fixed by storing the result as a pointer in the response struct, so successful zero values are still included.

6.2. Part 2 Challenges

The main challenge in Part 2 was designing an experiment that showed meaningful performance differences. Since the VM had only 2 vCPUs, runtime.NumCPU() was equal to 2. To make the comparison more useful, GOMAXPROCS=4 was added as an oversubscription case.

Another challenge was generating charts without external libraries. This was solved by implementing a simple SVG chart generator in Go using only standard-library packages.

6.3. Part 3 Challenges

In Part 3, the Docker image initially needed could not be get from the internet, so we used the local (intranet available) DARVAGcloud container registry:

- docker.darvagcloud.com/library/golang:1.22-bookworm
- docker.darvagcloud.com/library/debian:bookworm-slim

After fixing the image paths, the Docker image built successfully.

Another challenge was proving access from outside the VM. This was solved by configuring a Sangfor firewall DNAT rule from 89.38.215.235:18080 to 192.168.1.13:8080. External curl outputs and container logs confirmed that the service was reachable.

7. Conclusion

This homework implemented three connected systems concepts.

Part 1 demonstrated inter-process communication between two independent Go programs using Linux named pipes. The Worker process stayed alive and processed multiple sequential requests.

Part 2 demonstrated that increasing concurrency does not always improve performance. On the 2-vCPU VM, GOMAXPROCS=2 generally improved CPU-bound throughput compared with GOMAXPROCS=1, while GOMAXPROCS=4 did not provide proportional improvement. High goroutine counts increased latency, especially when synchronization was involved.

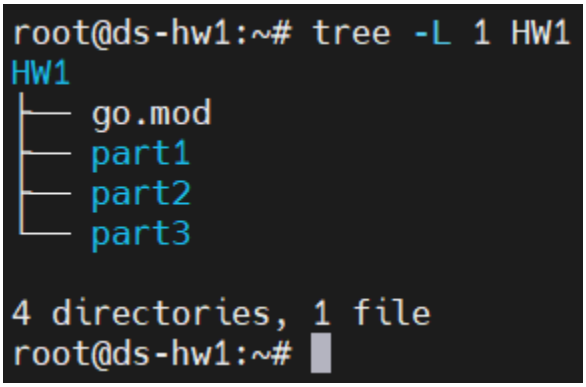
Part 3 converted the computation service into an HTTP service, packaged it as a Docker container, ran it inside the VM, and exposed it externally using DNAT. The external curl outputs and container logs confirmed that requests from outside the VM reached the containerized service.

Overall, the project demonstrates:

- Difference between independent processes and goroutines
- IPC design using named pipes
- Request/response protocol design
- Concurrent benchmark design
- Go scheduler and GOMAXPROCS behavior
- Docker containerization
- VM networking and host-to-guest service exposure

8. Final Submission Notes

The final submitted folder should be:



With report.pdf also under HW1

Also compiled binaries are intentionally kept.